

EE533_Lab8

1. Convertible FIFO Design

Interface with NetFPGA and Process:

```
// NetFPGA interface
input [63:0] in_data,
input [7:0] in_ctrl,
input in_wr,
output reg in_rdy,
output reg [63:0] out_data,
output reg [7:0] out_ctrl,
output reg out_wr,
input out_rdy,

// Processor interface
input [63:0] proc_data_in,
input [7:0] proc_addr_in,
output reg [63:0] proc_data_out,
input proc_web,
input proc_memEn,
input pin_di, //processor data process request
output reg p_en, //processor enable
```

Pin_di is a input signal sent by the processor indicating that processor is ready to process data.

FSM:

```
// FSM state definitions
parameter IDLE = 4'b0001,
          RX   = 4'b0010,
          // PROCESS = 4'b0100,
          EX   = 4'b0100,
          TX   = 4'b1000;

// FSM states
case (state)
IDLE: begin
    in_rdy = 1'b1;
    if (in_wr_reg) begin
        next_state = RX;
        web = 1'b1;
    end else next_state = IDLE;
end
RX: begin
    in_rdy = ~control_nonzero;
    if (in_wr_reg) begin
        web = 1'b1;
        next_state = control_nonzero ? EX : RX; //if 1, end of the packet, turn to ex
    end else next_state = RX;
end
EX: begin
    p_en = ~p_di;
    addrb = proc_addr_in;
    enb = proc_memEn;
    web = proc_web;
    dinb = {8{1'b0}, proc_data_in};
    proc_data_out = doutb[63:0];
    next_state = p_di ? TX : EX;
end
TX: begin
    if (out_rdy) begin
        out_wr_w = 1'b1;
        next_state = (count == 8'd1) ? IDLE : TX;
    end else next_state = TX;
end
end
```

```

// FSM sequential logic
always @(posedge clk or negedge reset) begin
    if (~reset) begin
        state <= IDLE;
        count <= 0;
        wp <= 0;
        rp <= 0;
        packet_count <= 0;
    end else begin
        state <= next_state;
        // Update counters and pointers based on state
        case (state)
            IDLE: if (in_wr_reg) begin
                count <= count + 1;
                wp <= wp + 1;
                packet_count <= packet_count + 1;
            end
            RX: if (in_wr_reg) begin
                count <= count + 1;
                wp <= wp + 1;
                packet_count <= packet_count + 1;
            end
            TX: if (out_rdy) begin
                rp <= rp + 1;
                count <= count - 1;
            end
        endcase
    end
end
end

```

Spinate 2.10.5 Free (Micro)

Dual-port SRAM module

```

// Memory block instance
Memblock fifol(
    .addra(memAddressQuery),
    .addrb(addrb),
    .clka(clk),
    .clkb(clk),
    .dina(72'b0),
    .dinb(dinb),
    .douta(memDataOut),
    .doutb(doutb),
    .enb(enb),
    .wea(1'b0),
    .web(web)
);

```

2. Simulation of FIFO Module